# Response to Reviewers — Paper 367005

**Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study**

We thank the reviewers and the programme committee. As the committee suggested, we answer the overlapping comments by theme: benchmark leakage, metric validity, threshold calibration, domain heterogeneity and prompt sensitivity. While preparing the revision we also found errors in the submitted version; they are listed in a separate section at the end, because two of them change reported numbers. All analyses added in the revision were planned after the reviews and are labelled as exploratory in the paper. Section numbers refer to the revised manuscript.

## 1. Benchmark leakage from reverse-engineered stories (R1, R3; tracker B1)

**Comment.** The user stories were reverse-engineered from the expert feature files, which may create circular lexical or structural leakage.

**Response.** We agree that this is the main threat to the study.

- §3.1 now describes exactly how the stories were made. GPT-5.4-mini (temperature 0) wrote each story from the first 15,000 characters of its reference file, which cut 84 of the 100 Fineract files. The model that wrote the stories is also the model under test, and §6.1 says so.
- §4.5 measures lexical leakage. A median of 4.9% of the distinct word trigrams of a reference also occur in its story. This overlap correlates with skeleton cosine in Sylius ($\rho = 0.29$, $p = 0.003$) and Diaspora ($\rho = 0.33$, $p = 0.009$), but not in Fineract ($\rho = 0.11$, $p = 0.30$).
- We had no independently written backlog. We therefore restrict all conclusions to reverse-engineered stories and state in §6.1 that results may overstate performance on natural requirements. The trigram measure captures lexical but not structural leakage, which we also state.
- The three prompt exemplars (ids 2, 102, 202) were part of the evaluated set. They are now excluded from the primary analysis ($N = 258$), and the $N = 261$ results are reported alongside.

## 2. Metric validity: critical changes and Given–When–Then dependencies (R1, R2, R3; tracker B2, H3)

**Comment.** SBERT cosine can miss operator, negation and numeric errors, and does not verify temporal or causal dependencies.

**Response.** We agree, and the new evidence shows the problem is larger than the comment suggests.

- §3.4 now describes the skeleton exactly as implemented. The submitted text wrongly said that structural keywords were removed, which R2's comment reflects; they are kept (item 3 in Section 6). Numbers and quoted values are masked, so the metric is blind

to changed numbers by construction. The encoder also truncates at 256 tokens, which affects 174 of the 258 reference skeletons.

- **Defect audit (§3.6, §4.4).** We sampled 60 files with cosine ≥ 0.85 (20 per project) and labelled each against its acceptance criteria, with a business-logic facet and a temporal/causal facet. 25 of the 60 files contain a defect of severity 2 or 3 (Sylius 6/20, Fineract 16/20, Diaspora 3/20). The most frequent defects are wrong expected outcomes and outcomes that no Then step verifies. The two facets flag the same 25 files, so we report them as one result.
- **How the labels were produced.** An AI assistant (Codex) drafted the labels and an author reviewed and retained all of them, recorded as one confirmation. The first author, who reports not having seen the labels or their aggregate rates before rating, labelled 15 of the files independently, using AI-prepared reading aids that contained no labels (Disclosure section): Cohen's $\kappa = 0.571$ (business logic) and 0.609 (temporal/causal). Both are below the 0.7 our protocol required. We did not run the consensus round the protocol foresaw, and we report this deviation and the unchanged labels in §4.4 and §6.2.
- **Perturbation test (§4.4).** Seeded defects in 623 valid scenario pairs never lowered cosine below 0.8995. Swapped or reordered steps scored at least 0.954, although our keyword-order check flagged all of them.
- **Structural check (§4.4).** A keyword-order check flags 5 of 255 parseable generated files and 40 of 244 parseable references. Correct order did not imply correct content.

## 3. Threshold calibration (R1, R3; tracker B3)

**Comment.** The 0.85 threshold lacks calibration, and the median barely exceeds it.

**Response.** We agree. §3.4 now states that the threshold came from the proposal and was not calibrated. The pilot used to set it (30 stories, mean 0.859) is part of the evaluated set.

- **Sensitivity (§4.3; Table 6 in Appendix A).** The median is significantly above every threshold up to 0.84 ($p = 0.0097$) and above none from 0.85 upwards ($p = 0.474$ at 0.85). At every threshold Sylius has the largest share of files above it; Fineract has the smallest from 0.83 upwards, and Diaspora below that.
- **Calibration attempt.** Calibration against the audit labels was not possible, because the audit covered only files ≥ 0.85. Within that range, cosine did not differ significantly between defective and non-defective files (Mann–Whitney $p = 0.21$, AUC 0.40). With 60 files from a restricted range, this neither shows nor rules out that the metric discriminates.
- Following the tracker, we softened the claims. 0.85 is now a descriptive reference point for structural similarity, and the paper no longer reads a score above it as correct (§5.2).

## 4. Domain heterogeneity (R2; tracker H1)

**Comment.** The pooled result masks failure in two of three domains.

**Response.** We agree. One small correction to the comment: with the recomputed data, the two projects below 0.85 are Fineract (median 0.826) and Diaspora (0.831). Sylius (0.887) is above it.

- §4.2 and Table 3 now lead with the per-project results. Fineract and Diaspora are significantly below 0.85 (one-sided $p < 0.001$ and $p = 0.008$).
- The projects differ (Kruskal–Wallis $H = 60.87$, $p < 0.001$, $\eta^2_H = 0.231$). Sylius differs from both other projects; the difference between Fineract and Diaspora is not significant (Holm $p = 0.74$).
- The abstract, discussion and conclusion no longer claim a general capability from the pooled median.

## 5. Prompt sensitivity and syntax compliance (R2; tracker H2, M1)

**Comment.** There is a single hand-picked exemplar per project and no zero-shot or alternative-exemplar condition. The high syntax compliance may come from the exemplar.

**Response.** We added two conditions on the full set, with the same model, temperature, system messages and clean-up as the original run (§3.2):

- C1 is zero-shot.
- C2 uses a different exemplar per project, drawn with a fixed seed before any output existed.

The analysis plan was written before the runs and is in the package with its SHA-256 hash. It specifies paired Wilcoxon and exact McNemar tests against C0 on the 255 files that are not an exemplar in any condition, with Holm correction. Results (§4.7, Table 4):

- **Zero-shot (C1)** lowered similarity in 182 of 255 files: median 0.819 instead of 0.853 (paired Wilcoxon, Holm $p < 0.001$).
- **Other exemplars (C2)** raised the median to 0.875, which is significantly above 0.85 (one-sided $p < 0.001$). The RQ1 decision therefore depends on the exemplars, mostly through Fineract (median 0.827 – 0.876). We report this in the abstract, results, discussion and conclusion, and keep C0 as the primary result as the plan specified.
- **Parser compliance stayed high under all three prompts:** 252, 252 and 254 of 255 files parse under C0, C1 and C2 (exact McNemar, Holm $p = 1.0$). For M1, this means the high compliance also holds without an exemplar; the tests do not establish equivalence. §3.5 now separates grammar compliance from semantic correctness explicitly: a parser pass means only that a file is valid Gherkin.

## 6. Corrections to the submitted version

We found the following errors while preparing the revision. We report them so that readers of the submitted version can see what changed.

1. **Parser compliance was overstated (RQ2).** The submitted check ran behave in dry-run mode (`--dry-run`) without step definitions. Behave then stops with a configuration error before parsing, and the script counted this as a PASS, even for plain text. With the parser itself, 255 of 258 files pass (98.84%, CI [96.64, 99.76]); with all 261 files, 258 pass. The RQ2 decision is unchanged.
2. **Table 3 of the submitted version did not match the data.** The recomputed values are a minimum of 0.585, Q1 0.805, Q3 0.890, a maximum of 0.994, and a mean of 0.848 ±

0.068 (N = 258). We could not reproduce Cliff's δ = 0.038. We replaced it with δ = P(x > 0.85) − P(x < 0.85), which gives 0.054 (0.065 for N = 261). The median and the RQ1 decision are unchanged.

3. **Skeleton description.** The paper said structural keywords were removed. In fact they are kept, and the skeleton masks values and drops non-keyword lines.
4. **Unreported encoder truncation** at 256 tokens (see Section 2 above).
5. **Story construction** was not described (see Section 1 above).
6. **The exemplars were in the evaluated set** (see Section 1 above).
7. **The threshold pilot overlapped with the evaluated set** (see §3 above).
8. **Domain spread.** The statement that the social-network subset was more consistent was wrong. Its IQR (0.087) is the widest.
9. **The API log** records the requested rather than the returned model name, and its cost used outdated prices. Recomputed from the logged tokens at the Standard list price, the run cost an estimated $1.96.
10. **Not all reference files parse.** The paper said each source scenario was required to parse. In fact 14 of the 261 references fail with behave 1.3.3 (13 with 1.2.6).
11. **Unit of analysis.** It is now stated as the feature file. The 258 analysed files contain 2,554 reference and 1,913 generated scenarios.
12. **Hypotheses.** The proposal stated them for the mean. The paper states them for the median and names the symmetry assumption under which the signed-rank test addresses the median.
13. **Output cap.** The proposal planned a 1,500-token cap that the run did not apply. This deviation is now declared.
14. **References.** The DOI of the Clopper–Pearson reference was added.

All scripts, inputs and outputs for these corrections are in the replication package under `revision/`.